

Com base no estado atual do Dream Team Governance Engine (DTGE) e nos protocolos estabelecidos, aqui estão 20 perguntas fundamentais sobre o andamento do projeto e suas respectivas soluções técnicas e de governança:

Implementação e Componentes Faltantes

- **Qual é a prioridade imediata para completar o fluxo de governança?**
- **Solução:** Construir o agente **02_Qualificador_Intencao**, que é o "elo que falta" para conectar o Coletor de Continuidade ao Orquestrador 1, 2.
- **O que falta para o Agente 01_Coletor_Continuidade entrar em operação?**
- **Solução:** O contrato já existe, mas é necessário realizar a **canonização formal** e desenvolver o **código real** do agente 1, 3.
- **Como garantir que os agentes funcionem em conjunto e não apenas isoladamente?**
- **Solução:** Realizar um **teste de integração ponta a ponta** onde os 7 agentes reais chamem uns aos outros em sequência, algo que ainda não foi executado 1, 2.
- **Existem lacunas identificadas no código atual do Orquestrador?**
- **Solução:** Sim. É necessário implementar a **captura de criticidade**, a checagem **anti-drift** e a lógica de **Pausa/Análise de Opções** que constam no contrato, mas não no código atual 1, 4.
- **Qual o procedimento para adicionar uma nova funcionalidade ao sistema?**
- **Solução:** Seguir a regra de **"Contratos antes de código"**: nenhuma mudança crítica de execução deve ocorrer sem a formalização e canonização do contrato do agente 5, 6.

Segurança e Integridade

- **Como o sistema impede que um erro entre em loop infinito?**
- **Solução:** O Sentinela e o Orquestrador monitoram a **profundidade_ciclo**. Se exceder **3 níveis** ou houver repetição exata de (agente, parâmetros), o sistema gera um **ALERTA_INTEGRIDADE** e bloqueia a ação 7, 8.
- **Como garantir que os scripts executados no sandbox não foram alterados maliciosamente?**
- **Solução:** Utilizar a **camada de blindagem** (**validar_integridade**), que compara o hash SHA-256 do arquivo com o manifesto de confiança registrado na cadeia imutável do Registrador 9, 10.
- **Como prevenir ataques de injeção de comandos via argumentos?**
- **Solução:** Aplicar uma **lista branca (white-list)** estrita de caracteres permitidos e usar a função **executar_seguro()**, que proíbe o uso de **shell=True** 11-13.
- **O sistema de identidade de governança é infalível?**
- **Solução:** Não. Atualmente, o campo **autorizado_por** é declarativo e não uma assinatura criptográfica. A solução futura é implementar um **sistema de identidade real** para elevar a barra de segurança 11.
- **Como detectar se o histórico de registros (auditoria) foi violado?**
- **Solução:** O Registrador mantém uma **cadeia de hashes encadeada** (tamper-evident). A função **verificar_integridade_cadeia()** recalcula os hashes para detectar qualquer alteração retroativa 14, 15.

Governança e Decisão

- **Os agentes podem tomar decisões executivas sozinhos?**
- **Solução:** Não. O "**Gate Humano**" é obrigatório; nenhuma AUTORIZACAO_EXECUCAO (TOOL-EVT-005) é gerada sem um comando explícito do humano 5, 16.
- **Como evitar a criação de componentes redundantes (desvio estrutural)?**
- **Solução:** Aplicar a regra **Anti-Drift**: antes de propor algo novo, o Arquiteto e o Orquestrador devem consultar a pasta **_BASE/CANONIZACOES/** e priorizar a extensão de ferramentas existentes 4, 17, 18.
- **O que fazer se uma tarefa for classificada como CRÍTICA?**
- **Solução:** O Plano de Ação deve obrigatoriamente incluir um **requisito de aprovação humana explícita**. Se não houver, o Sentinela bloqueia o avanço 4, 19, 20.
- **Qual a diferença entre o token "SEGUIR" e uma autorização real?**
- **Solução:** **SEGUIR** é um avanço cognitivo (não-executivo). Apenas o token **AUTORIZACAO_EXECUCAO** permite que o Executor realize mudanças reais no sandbox 5, 21.
- **Como o sistema lida com situações onde há múltiplos caminhos para uma solução?**
- **Solução:** O Orquestrador gera uma **Análise de Opções** (TOOL-COG-001), onde o Analista compara trade-offs e recomenda a melhor opção para decisão humana 7, 22.

Operação e Continuidade

- **Como o projeto mantém a eficiência no uso de tokens em conversas longas?**
- **Solução:** Uso de **JSON compacto**, logs que guardam apenas o **hash dos argumentos** (não o dado inteiro) e referenciamento de templates fixos por hash 23-25.
- **O que acontece se uma execução autorizada falhar?**
- **Solução:** O Executor registra a falha e aciona o plano de **Rollback**, que deve ter sido modelado pelo Arquiteto desde a origem da tarefa 26-28.
- **Como garantir que o Rollback não cause mais danos ou apague dados?**
- **Solução:** O Rollback segue a regra de ser **sempre aditivo**: ele nunca sobrescreve o erro, mas encadeia a correção como uma nova versão (prev_ref) 27, 29.
- **Como retomar o projeto em uma nova sessão sem perder o contexto?**
- **Solução:** Utilizar o arquivo **MEMORIA_DREAMTEAM_v1.md** e o **INDICE.jsonl** como fontes de verdade para o estado dos agentes e a linha do tempo de decisões 2, 30.
- **Qual o status atual dos contratos e códigos?**
- **Solução:** Os 7 agentes principais estão **congelados e canonizados** (contratos e códigos). As pendências agora focam nos agentes de entrada (Qualificador e Coletor) 3, 31.